

dbt Training

● Beginner · Module 05 · 90 min

Sources and the Medallion Architecture

How dbt knows about Bronze tables, why layer boundaries matter, and how source freshness protects your pipelines.

The Medallion Architecture

```

Flowchart TD
    SRC(["Source Systems - HubSpot · Shopify · APIs"])
    LAMBDA["AWS Lambda - Ingestion"]
    BRONZE["BRONZE - raw · append-only - Lambda owns"]
    STAGING["STAGING - views · stg_hubspot_*"]
    SILVER["SILVER - dim_* · fct_* · bridge_*"]
    GOLD["GOLD - mrt_*"]
    PBI(["Power BI"])

    SRC --> LAMBDA
    LAMBDA --> BRONZE
    BRONZE -->|"source()"| STAGING
    STAGING -->|"ref()"| SILVER
    SILVER -->|"ref()"| GOLD
    GOLD --> PBI

    classDef dbt fill:#ecfdf5,stroke:#16a34a,color:#065f46
    classDef lambda fill:#fef9c3,stroke:#ca8a04,color:#713f12
    classDef bronze fill:#f1f5f9,stroke:#94a3b8,color:#475569
    class STAGING,SILVER,GOLD dbt
    class LAMBDA lambda
    class BRONZE bronze
  
```

dbt references Bronze as a **source**, not via `ref()`. Bronze tables are never built by dbt.

When referencing Bronze in a staging model

```
{{ source('hubspot', 'contacts') }}
```

Declares a dependency on the source → DAG tracks it

When referencing any other dbt model

```
{{ ref('dim_patient') }}
```

Resolves to correct schema per target environment

Declaring Sources in `sources.yml`

```
```yaml {all|1-7|9-16|18-20|all}
```

```
version: 2
```

```
sources:
```

```
- name: hubspot # alias used in {{ source() }}
```

```
database: BRONZE
```

schema: HUBSPOT

description: "HubSpot CRM data ingested via AWS Lambda."

```
tables:
 - name: contacts
 description: "One row per HubSpot contact. Append-only."
 loaded_at_field: _ingested_at # column dbt uses for freshness

 - name: deals
 description: "HubSpot deal records including pipeline stage history."
 loaded_at_field: _ingested_at

 - name: pipeline_stages
 description: "Static lookup: pipeline stage definitions."
 # no loaded_at_field – static table, skip freshness
```

```
<!--
```

Use line highlights: first show the source-level config (name, database, schema), then the table declarations, then the pipeline\_stages table without a loaded\_at\_field.

Without this file, `{{ source('hubspot', 'contacts') }}` fails at the Parse phase – dbt can't resolve the source.

The loaded\_at\_field is the column dbt queries to check freshness: `SELECT MAX(_ingested_at) FROM BRONZE.HUBSPOT.contacts`. If the MAX is older than your error threshold, the freshness check fails.

Static tables like pipeline\_stages don't need freshness – they change infrequently and deliberately. Opting out with freshness: null prevents false alerts.

```
-->
```

```

```

```
Hardcoding vs. `{{ source() }}`
```

```
<div class="grid grid-cols-2 gap-6 mt-4">
```

```
<div>
```

```
** ❌ Hardcoded – never do this**
```

```
```sql
```

```
SELECT *
```

```
FROM BRONZE.HUBSPOT.contacts
```

- Does not appear in the DAG
- Freshness check doesn't work

- Always points to prod — ignores your dev target
- Schema change requires updating every model
- **  With `source()` — always do this**

```
SELECT *
FROM {{ source('hubspot', 'contacts') }}
```

- Appears in DAG with lineage
- Freshness check works
- Resolves to correct schema per target
- Schema change: update sources.yml once

Both produce the same compiled SQL in practice — but they're not equivalent. You lose DAG visibility, freshness, and environment-awareness with hardcoding.

Source Freshness

```
sources:
  - name: hubspot
    freshness:
      warn_after: {count: 6, period: hour}
      error_after: {count: 24, period: hour}

  tables:
    - name: contacts
      loaded_at_field: _ingested_at

    - name: pipeline_stages
      freshness: null          # static table – opt out
```

```
dbt source freshness
```

```
Found 1 source, 2 tables.
contacts: 3 hours 42 minutes ago – PASS
deals:    26 hours 15 minutes ago – ERROR
```

In your orchestrator: Freshness check runs before any dbt models. If a source errors, the pipeline stops — preventing Silver and Gold from being built on stale Bronze data.

Exercise: Add a New Source (25 min)

Scenario: Adding the HubSpot `owners` table — `BRONZE.HUBSPOT.owners` . Updated every 12 hours. Has a `_ingested_at` column.

Step 1 — Add to `sources.yml`

Add the `owners` table with freshness thresholds: warn at 14h, error at 25h.

Step 2 — Write the staging model

Create `stg_hubspot__owners.sql` : reference source correctly, select `owner_id` , `first_name` , `last_name` , `email` , rename `_ingested_at` → `ingested_at` , materialise as view.

Step 3 — Verify

Run `dbt compile --select stg_hubspot__owners` . Verify the compiled output references `BRONZE.HUBSPOT.owners` .

Step 4 — Read the docs

Google dbt docs about source configuration: `dbt docs add sources to your dag` . Browse, note diverse options.

What You Can Now Do — Module 05

✓

Medallion Architecture

Explain Bronze → Staging → Silver → Gold and why dbt never writes to Bronze

✓

Declare Sources

Write a `sources.yml` with source name, database, schema, and table entries

✓

Use `source()` Correctly

Reference Bronze via `{{ source() }}` — never hardcode — and explain what you lose without it

✓

Source Freshness

Configure `warn_after` / `error_after` thresholds and interpret `dbt source freshness` output

Tier 1 checkpoint: You can read from any Bronze source, declare it correctly, and protect the pipeline from stale data — before a single Silver model runs.

layout: default

background: '#f9f8f5'

Module 05 Complete

Next: Module 06

Testing Data Quality

Prep Q1: What must exist before using `{{ source('hubspot', 'contacts') }}`?

Prep Q2: Two things lost by hardcoding vs `source()`?

Prep Q3: What column does dbt query for freshness?

Prep Q4: Why does dbt NOT own the Bronze layer?